

MANUAL DE CÓDIGO

Sistema Integral de Gestión Educativa Académica (SIGEA)

1. Introducción

Este manual documenta, a nivel de código fuente, el sistema SIGEA desarrollado hasta la fecha para automatizar la gestión de notas, asistencia y comunicados a padres de familia de una institución educativa, correspondiente a la etapa de Desarrollo y Codificación del plan de trabajo (actividades 9 a 13).

Pila tecnológica: PHP 8 procedural con PDO (prepared statements), MySQL/MariaDB 8 tablas relacionadas con integridad referencial, HTML5/CSS3/JavaScript en el frontend, servidor XAMPP. Total de código PHP entregado a la fecha: 3,751 líneas distribuidas en 33 archivos .php.

2. Requisitos e instalación

- PHP 8.0 o superior con extensión PDO MySQL.
- MySQL o MariaDB (compatible con XAMPP).
- Servidor web Apache (incluido en XAMPP).
- Importar el esquema desde portal_notas.sql (crea la base portal_notas con 8 tablas y datos de prueba).
- Configurar credenciales en config/conexion.php: \$DB_HOST, \$DB_NAME, \$DB_USER, \$DB_PASS.
- Colocar el proyecto dentro de htdocs (XAMPP) y acceder vía `http://localhost/<carpeta>/`.

Credenciales de prueba sembradas en el script SQL — Usuario: admin / Contraseña: Admin123! (rol administrador/secretaría).

3. Arquitectura general

El sistema usa una arquitectura de scripts PHP procedurales organizados por rol de usuario (administrador, profesor, padre de familia), con una capa de conexión a base de datos centralizada (PDO, config/conexion.php) y funciones de control de sesión compartidas (includes/sesion.php). Cada página de un rol repite el mismo patrón: (1) incluir sesion.php y conexion.php, (2) invocar la función de guardia correspondiente, (3) procesar POST/GET si aplica con sentencias preparadas, (4) renderizar la vista HTML embebida con datos escapados mediante `h()`.

Carpeta / archivo	Responsabilidad
config/conexion.php	Conexión PDO a MySQL (host, usuario, contraseña, base portal_notas), modo de errores por excepción, fetch por defecto asociativo.
includes/sesion.php	Control de sesión y funciones de guardia de acceso: <code>exigirAdmin()</code> , <code>exigirPadre()</code> , <code>exigirProfesor()</code> ; función <code>h()</code> de escape HTML.
includes/sidebar_admin.php / sidebar_padre.php / sidebar_profesor.php	Menús de navegación lateral específicos por rol.
includes/topbar.php	Barra superior común de las vistas internas (título de página dinámico).
login.php / index.php / logout.php	Autenticación contra administradores, enrutamiento inicial según sesión activa y cierre de sesión.
admin/*.php (8 archivos)	Panel y CRUD completos para el rol administrador: dashboard, estudiantes, materias, notas, asistencia, avisos, padres, profesores.
profesor/*.php (6 archivos)	Vistas y lógica del rol profesor: dashboard, estudiantes, notas, asistencia, conducta, avisos.

Carpeta / archivo	Responsabilidad
padres/*.php (4 archivos)	Vistas de consulta de solo lectura del rol padre de familia: dashboard, notas, asistencia, avisos.
assets/css/style.css	Hoja de estilos general del sistema (sin frameworks externos de CSS).
assets/js/app.js (106 líneas)	Lógica JavaScript de interacción en el frontend.
portal_notas.sql	Script de creación de la base de datos (DDL) y datos de prueba (DML).

4. Modelo de base de datos

El script `portal_notas.sql` define 8 tablas en motor InnoDB con claves foráneas e integridad referencial (ON DELETE CASCADE / SET NULL):

Tabla	Columnas principales	Relaciones / restricciones
administradores	id, nombre, usuario (único), password, correo, creado_en	Usada tanto para administradores como, según el código de profesor/, para profesores.
padres	id, nombre, apellido, correo, telefono, usuario (único), password, creado_en	Un padre puede tener varios estudiantes.
estudiantes	id, carnet (único), nombre, apellido, grado, seccion, fecha_nacimiento, id_padre	FK id_padre → padres.id ON DELETE CASCADE.
materias	id, nombre, docente, grado	
periodos	id, nombre, activo	
notas	id, id_estudiante, id_materia, id_periodo, nota (0–10), comentario, fecha_registro	3 FKs en cascada; UNIQUE(estudiante, materia, periodo); CHECK nota entre 0 y 10.
asistencia	id, id_estudiante, fecha, estado (ENUM Presente/Ausente/Tardanza), observacion	FK a estudiantes; UNIQUE(estudiante, fecha) → evita duplicar asistencia del mismo día.
avisos	id, titulo, contenido, id_admin, fecha	FK id_admin → administradores.id ON DELETE SET NULL.

5. Autenticación y control de acceso

La autenticación (`login.php`) valida usuario y contraseña contra la tabla `administradores` usando `password_verify()` (`bcrypt`) y regenera el identificador de sesión al iniciar sesión (`session_regenerate_id`) como medida contra fijación de sesión.

```
$stmt = $pdo->prepare('SELECT * FROM administradores WHERE usuario = ?');
$stmt->execute([$usuario]);
$admin = $stmt->fetch();
if ($admin && password_verify($password, $admin['password'])) {
    session_regenerate_id(true);
    $_SESSION['admin_id'] = $admin['id'];
    $_SESSION['admin_nombre'] = $admin['nombre'];
    header('Location: dashboard.php');
}
```

El control de acceso por rol se centraliza en `includes/sesion.php` mediante tres funciones de guardia que cada página protegida invoca al inicio:

```
function exigirAdmin(): void { ... if (empty($_SESSION['admin_id'])) redirige a login ... }
function exigirPadre(): void { ... if (empty($_SESSION['padre_id'])) redirige a login ... }
```

```
function exigirProfesor(): void { ... exige admin_id + $_SESSION['admin_rol'] === 'profesor' ... }
```

Todas las páginas que muestran datos provenientes del usuario o de la base de datos utilizan la función `h()` para escapar la salida HTML y mitigar XSS.

6. Detalle de módulos por rol

6.1 Rol administrador (admin/)

Archivo	Guardia	Tabla(s) BD	Operaciones implementadas
dashboard.php	exigirAdmin()	estudiantes, padres, materias, notas, avisos	4 SELECT de conteo/promedio + últimos avisos y últimas notas registradas (JOIN).
estudiantes.php	exigirAdmin()	estudiantes, padres	SELECT, INSERT, UPDATE, DELETE. Combo de padres disponibles; JOIN para mostrar nombre del padre.
materias.php	exigirAdmin()	materias	SELECT, INSERT, UPDATE, DELETE (nombre, docente, grado).
notas.php	exigirAdmin()	notas, estudiantes, materias, periodos	INSERT/UPDATE combinado con ON DUPLICATE KEY UPDATE (upsert de nota por estudiante+materia+periodo); creación de nuevos periodos; DELETE.
asistencia.php	exigirAdmin()	asistencia, estudiantes	Registro por upsert (ON DUPLICATE KEY UPDATE) según estado (Presente/Ausente/Tardanza); listado de últimos 100 registros; DELETE.
avisos.php	exigirAdmin()	avisos, administradores	INSERT de nuevo aviso asociado al admin en sesión; listado con JOIN al nombre del autor; DELETE.
padres.php	exigirAdmin()	padres	SELECT, INSERT, UPDATE (con o sin cambio de contraseña), DELETE.
profesores.php	valida \$_SESSION['admin_rol'] === 'admin'	administradores	Gestión de cuentas de profesor: valida duplicados de usuario/correo antes de insertar; INSERT con columnas rol, especialidad, telefono; UPDATE con o sin cambio de contraseña; DELETE restringido a rol='profesor'.

6.2 Rol profesor (profesor/)

Archivo	Guardia usada en el archivo	Tabla(s) BD	Operaciones implementadas
dashboard.php	exigirProfesor()	administradores	Datos del profesor en sesión (nombre, correo, especialidad).
estudiantes.php	exigirProfesor()	materias, estudiantes	Filtra estudiantes por grado según las materias que imparte el docente (LIKE sobre columna docente).
notas.php	exigirAdmin() 	notas, estudiantes, materias, periodos	Registro de notas por upsert; DELETE condicionado a la materia del profesor.
asistencia.php	exigirAdmin() 	asistencia, estudiantes, materias	Registro masivo de asistencia por grado con upsert.

Archivo	Guardia usada en el archivo	Tabla(s) BD	Operaciones implementadas
conducta.php	exigirAdmin() Δ	conducta (no existe en portal_notas.sql) Δ	INSERT / SELECT / DELETE sobre una tabla conducta que no está definida en el script de base de datos entregado.
avisos.php	exigirAdmin() Δ	avisos, administradores	Listado de avisos (solo lectura en este archivo).

6.3 Rol padre de familia (padres/, solo lectura)

Archivo	Guardia	Tabla(s) BD	Operaciones implementadas
dashboard.php	exigirPadre()	estudiantes, notas, avisos	Lista los hijos del padre en sesión, promedio de notas por hijo, últimos 3 avisos.
notas.php	exigirPadre()	notas, estudiantes, materias, periodos	Consulta de notas de los hijos del padre en sesión (JOIN múltiple).
asistencia.php	exigirPadre()	asistencia, estudiantes	Consulta del historial de asistencia de los hijos del padre en sesión.
avisos.php	exigirPadre()	avisos	Listado de todos los avisos publicados (solo lectura).

7. Hallazgos de revisión

Durante la elaboración de este manual se revisó el código fuente sin modificarlo, conforme a la instrucción recibida. Se documentan a continuación las inconsistencias detectadas, útiles como insumo para la próxima etapa de pruebas (actividad 14 del plan de trabajo):

- Rol de sesión no asignado en el login: login.php únicamente guarda `$_SESSION['admin_id']` y `$_SESSION['admin_nombre']`; nunca asigna `$_SESSION['admin_rol']`. Sin embargo, `includes/sesion.php` (`exigirProfesor`) y varios archivos de `profesor/` y `admin/profesores.php` sí condicionan el acceso a ese valor de sesión, por lo que el flujo de acceso diferenciado `admin/profesor` podría no operar como se espera con el login actual.
- Guardia inconsistente en `profesor/`: los archivos `profesor/notas.php`, `profesor/asistencia.php`, `profesor/conducta.php` y `profesor/avisos.php` llaman a `exigirAdmin()` en vez de `exigirProfesor()` (a diferencia de `profesor/dashboard.php` y `profesor/estudiantes.php`, que sí usan `exigirProfesor()`).
- Tabla `conducta` no definida en `portal_notas.sql`: `profesor/conducta.php` realiza INSERT, SELECT y DELETE contra una tabla llamada `conducta` que no aparece entre las 8 tablas creadas por el script de base de datos entregado.
- Columnas usadas por `admin/profesores.php` no definidas en el esquema: el módulo inserta/actualiza los campos `rol`, `especialidad` y `telefono` en la tabla `administradores`, columnas que no están declaradas en el CREATE TABLE `administradores` de `portal_notas.sql` (que solo define `id`, `nombre`, `usuario`, `password`, `correo`, `creado_en`).
- Diferencia de nombres entre README.md y el código entregado: el README describe archivos `config/db.php`, `includes/auth.php`, `includes/functions.php`, `includes/header.php` e `includes/footer.php`, que no coinciden con los nombres reales del proyecto (`config/conexion.php`, `includes/sesion.php`, vistas embebidas en cada archivo de rol).